

Hard-Coding app.py in Python

The CLI entrypoint: argparse, logging, one-shot questions, and a REPL loop.

Project: logistics-rag-agent

File: app.py

Role: Human interface over agent.py and rag.py

app.py does not implement RAG or tools. It parses flags, configures logs, and calls ask_agent or ask_rag.

1. Big picture

```
python app.py "What is EOQ for ...?"
python app.py --rag-only -k 6 "Cold chain rule?"
python app.py -v
    then type questions at Q> until empty line / Ctrl-D
```

WHY

Separating CLI from library code (agent/rag/tools) keeps business logic importable without starting argparse.

2. Build step by step

Step 1: Imports

```
from __future__ import annotations

import argparse
import logging
import sys

from agent import ask_agent
from rag import TOP_K, ask as ask_rag
```

- ask as ask_rag -- rename so it does not clash with a local name 'ask'
- TOP_K -- reuse default for --rag-only -k help text and default value

WHY

Importing callables (ask_agent, ask_rag) keeps app.py thin: no Chroma setup here.

Step 2: configure_logging

```
def configure_logging(verbose: bool) -> None:
    level = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)s [%(name)s] %(message)s",
        datefmt="%H:%M:%S",
    )
    logging.getLogger("logistics_rag").setLevel(
        logging.DEBUG if verbose else logging.INFO
    )
```

- basicConfig -- root logging setup once at process start
- Explicit logistics_rag level -- ensures -v shows rag/tools/agent debug lines

WHY

Libraries log to 'logistics_rag'. Turning only root to INFO may still hide DEBUG unless that named logger is raised too -- hence the second setLevel.

Step 3: main(argv) signature

```
def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(...)
    ...
```

- argv=None -- production uses sys.argv; tests pass a fake list
- -> int -- process exit code (0 success)

WHY

Testable CLI: `main(['--rag-only', 'hello'])` without subprocess.

Step 4: Define CLI arguments

```
parser.add_argument("question", nargs="?", help="... or omit for REPL")
parser.add_argument("--rag-only", action="store_true", help="...")
parser.add_argument("-k", type=int, default=TOP_K, help="...")
parser.add_argument("-v", "--verbose", action="store_true", help="...")
args = parser.parse_args(argv)
```

- nargs='?' -- positional question is optional
- action='store_true' -- flag present => True, else False
- parse_args(argv) -- when argv is None, reads real command line

WHY

--rag-only is a teaching/debug switch: bypass tool routing and force pure grounded RAG.

Step 5: Choose answer backend

```
configure_logging(args.verbose)

def answer(question: str) -> str:
    if args.rag_only:
        return ask_rag(question, k=args.k)
    return ask_agent(question)
```

Nested function closes over args. One place for one-shot and REPL to call.

WHY

Closure beats repeating if args.rag_only in two places (DRY, fewer bugs).

Step 6: One-shot mode

```
if args.question:
    print(answer(args.question))
    return 0
```

WHY

Scripts and demos prefer: python app.py 'question' && echo success. Exit 0 signals OK to shells.

Step 7: REPL mode

```
print(f"Logistics RAG [{mode}] -- empty line or Ctrl-D to quit...")
while True:
    try:
        question = input("\nQ> ").strip()
    except (EOFError, KeyboardInterrupt):
        print()
        break
    if not question:
        break
    print(f"A> {answer(question)}")
return 0
```

- EOFError -- Ctrl-D (Unix) / Ctrl-Z Enter (Windows)
- KeyboardInterrupt -- Ctrl-C; still exit cleanly with code 0
- Empty string after strip -- polite quit without error

WHY

Catching both EOF and KeyboardInterrupt avoids ugly stack traces when a human exits.

Step 8: Script entrypoint

```
if __name__ == "__main__":  
    sys.exit(main())
```

- sys.exit(main()) -- forward the int return as the process status

WHY

Using raise SystemExit(main()) is equivalent; sys.exit is the common CLI style.

3. Full mental model of the repo

```
ingest.py -> build chroma_db from PDFs
prompts.py -> grounded prompt text
rag.py -> retrieve + prompt + chat (no tools)
tools.py -> EOQ / safety / SKU / doc-search tools
agent.py -> LLM picks tools, returns final text
app.py -> CLI over agent (default) or rag (--rag-only)
```

4. Practice

- python app.py -v "EOQ with demand 5000 order cost 40 holding 1.5"
- python app.py --rag-only "Summarize the SOP step for returns"
- python app.py # then ask two questions interactively

5. Common failures

- ModuleNotFoundError -- run from project root with rag-env activated
- No debug logs -- forgot -v
- Agent vs RAG confusion -- check whether --rag-only is set